

1. Konsep Dasar Functional Programming

Pertanyaan diskusi: kenapa FP dianggap cocok untuk scalable computing?

Jawaban singkat: kesulitan terbesar komputasi paralel adalah *shared mutable state*, yaitu data yang dipakai bareng dan bisa diubah oleh banyak proses. FP menghilangkan masalah itu dari akarnya: datanya immutable dan fungsinya pure. Akibatnya pekerjaan bisa dipecah, dijalankan di mana saja dan dalam urutan apa saja, lalu digabung lagi dengan hasil yang tetap sama.

1. Imperative vs functional

	Imperative	Functional
Fokus	<i>bagaimana</i> : urutan langkah	<i>apa</i> : transformasi data
Alat utama	variabel yang diubah, loop, assignment	fungsi, ekspresi, rekursi
State	disimpan di variabel, berubah terus	dibawa sebagai argumen, tidak diubah

```
// imperative: total diubah berkali-kali
var total = 0
for (x <- xs) total += x * x

// functional: data ditransformasi, tidak ada yang diubah
val total = xs.map(x => x * x).sum
```

2. Immutable data

Data tidak bisa diubah setelah dibuat. "Mengubah" artinya membuat versi baru, sedangkan versi lama tetap utuh.

```
v = jnp.array([2., 4., 6., 3.]) # contoh JAX dari fp-ssc-course
vu = v.at[2].set(7)           # vu = [2, 4, 7, 3]
v                             # tetap [2, 4, 6, 3]
```

Kenapa penting: tidak ada yang bisa mengubah data diam-diam, jadi data aman dibagi ke banyak thread tanpa lock.

3. Pure function

Output hanya ditentukan oleh input, dan fungsinya tidak punya efek samping: tidak mengubah variabel di luar, tidak mencetak, tidak membaca file, tidak memakai waktu atau bilangan acak.

```
def ll0(mu: Double)(x: Double): Double = -(x - mu)*(x - mu)/2.0 // pure

var hitung = 0
def f(x: Int): Int = { hitung += 1; x * 2 } // tidak pure, mengubah hitung
```

4. Referential transparency

Sebuah ekspresi bisa diganti dengan nilainya tanpa mengubah arti program. Ini konsekuensi langsung dari pure function.

- Kalau f pure, $f(3) + f(3)$ pasti sama dengan $2 * f(3)$.
- Kalau fungsinya tidak pure, misalnya `def g(): Int = { hitung += 1; hitung }`, maka $g() + g()$ bernilai $1 + 2 = 3$, sedangkan $2 * g()$ bernilai 2. Mengganti ekspresi dengan nilainya jadi tidak aman lagi.

Manfaatnya: hasil boleh di-cache (memoization), urutan eksekusi boleh diatur ulang, dan pemanggilan boleh dijalankan paralel. Kompiler atau library bisa melakukannya otomatis karena dijamin aman.

5. Higher-order function (HoF)

Fungsi yang menerima fungsi sebagai argumen atau mengembalikan fungsi. Contoh yang paling sering: `map`, `filter`, `reduce`, `fold`.

```
// dari ML-GA.scala: ascend menerima fungsi step, oneStep mengembalikan fungsi
def oneStep(learningRate: Double)(b0: DVD): DVD = b0 + learningRate*gll(b0)
val opt = ascend(oneStep(1e-6), init) // oneStep(1e-6) adalah fungsi DVD => DVD
```

Dengan HoF, pola seperti "lakukan ini ke setiap elemen" cukup ditulis sekali, lalu perilakunya diganti lewat fungsi yang dikirim.

6. Composition

Membangun hal besar dari fungsi-fungsi kecil yang disambung, seperti pipa.

```
val bersihkan: String => String = _.trim
val kecilkan: String => String = _.toLowerCase
val proses = bersihkan andThen kecilkan // proses(x) = kecilkan(bersihkan(x))
```

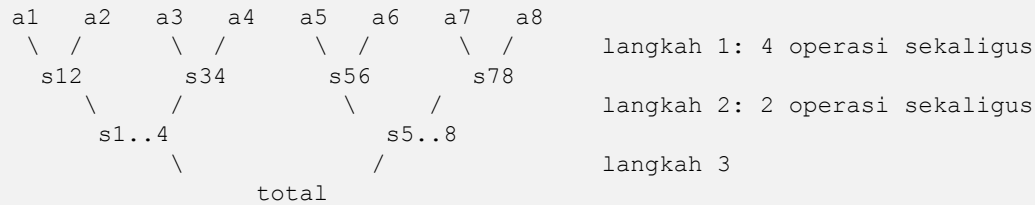
Tiap bagian kecil bisa dites sendiri. Kalau bagian-bagiannya benar, gabungannya juga benar. Menurut Intro fp-ssc-course, membangun model dan komputasi besar dari bagian kecil (divide and conquer) adalah cara yang bagus, dan cara ini sangat alami di FP.

7. Monoid dan map-reduce

Monoid = himpunan + operasi biner yang **asosiatif** + elemen **identitas**.

Himpunan	Operasi	Identitas
bilangan	+	0
bilangan	*	1
string	gabung	""
list	++	List()
bilangan	max	minus tak hingga

Syarat asosiatif $(a \cdot b) \cdot c = a \cdot (b \cdot c)$ ($\cdot$ adalah operasinya) artinya **cara pengelompokannya bebas**. Karena itu data bisa dipecah ke banyak prosesor lalu dijumlahkan dengan pola pohon (tree reduction):



3 langkah = $\log_2(8)$, bukan 7 langkah seperti penjumlahan satu per satu

Waktunya turun dari $O(n)$ ke $O(\log n)$ kalau prosesornya cukup banyak. Pengurangan tidak asosiatif ($(8-4)-2 \neq 8-(4-2)$), jadi tidak boleh di-reduce paralel.

Map-reduce = *map* tiap data ke sebuah monoid (bisa paralel), lalu *reduce* dengan operasi monoidnya (juga paralel). Namanya juga `foldMap`. Contoh di kursus: $\log\text{-likelihood}$ = jumlah $\text{ll}(x)$ untuk semua data, yaitu `map ll`, lalu `reduce +`.

```
// parallel.scala: kode sama, cukup tambah .par
(v map ll(0.0)) reduce (+_ ) // sekuensial, sekitar 5 detik
(vp map ll(0.0)) reduce (+_ ) // paralel, hasilnya sama persis: -4.844171665682075
```

Jadi, kenapa FP cocok untuk scalable computing?

1. **Tidak ada shared mutable state.** Race condition, lock, dan deadlock muncul karena banyak proses mengubah data yang sama. Dengan data immutable dan fungsi pure, masalah ini tidak ada, jadi paralelisme aman bahkan kadang otomatis.
2. **Referential transparency memberi kebebasan urutan.** Library bebas mengatur ulang, memparalelkan, mendistribusikan, bahkan menurunkan (autodiff) kode. JAX (`jit`, `grad`, `vmap`) dan Spark bekerja dengan cara ini.
3. **Monoid membuat reduce bisa dipecah.** Operasi asosiatif bisa dibagi ke banyak core atau banyak mesin lalu digabung, dan itulah inti map-reduce di Spark.
4. **Composition membuat kodenya skalabel.** Scalable bukan cuma soal cepat. Sistem besar juga lebih mudah dibangun dari bagian kecil yang sudah teruji, dan pure function paling gampang dites.

Batasan (supaya jawaban diskusinya berimbang)

- Membuat data baru terus berarti alokasi memori lebih banyak. Solusinya persistent data structure yang berbagi bagian yang sama (structural sharing), plus garbage collector.
- Program nyata tetap butuh I/O dan state. FP tidak menghapusnya, hanya mendorongnya ke pinggir program (misalnya `IO` di Haskell atau `cats-effect` di Scala).
- Paralelisme punya overhead, jadi untuk data kecil bisa malah lebih lambat.
- Kurva belajarnya lebih curam, dan tidak semua masalah enak ditulis secara fungsional.

Kaitan dengan yang sudah dicoba di fp-ssc-course

Konsep	Di mana terlihat
Rekursi pengganti loop	<code>logFact.scala: @tailrec, logFact 100000</code> tidak stack overflow
State dibawa sebagai argumen	<code>ML-GA.scala: fungsi go(b0, 110, itsLeft)</code> menggantikan <code>while + var</code>
Paralel tanpa lock	<code>parallel.scala (.par)</code> dan <code>futures.scala</code>
Hukum monoid dites	<code>ML-GA-laws.scala: law test Semigroup (asosiatif)</code> dan <code>Functor</code> , bagian dari 19 tes <code>sbt test</code> yang lolos
Immutable + autodiff	<code>JAX/ml-ga.py: grad(11)</code> dan <code>jit</code>